Advanced Big Data Analytics Project 2

Mobile Money Investigations with PostgreSQL and Neo4j

Issued 12 October 2026 Due 26 October 2026 at 11:59 PM CAT

Up to 3 students per group Total 100 marks

A mobile money provider in Rwanda wants its analysts to explain payment activity and identify accounts that need closer review. Daily reports show how much money moves through the service. To investigate an account, however, analysts must follow transfers between wallets and examine who owns those wallets or uses the same devices. Rapid onward payments or transfers returning to the starting account may justify further enquiries, while shared devices may have ordinary household or agent explanations.

Your group will act as those analysts. Use the September records to build the same reports and investigations in PostgreSQL and Neo4j. As the questions move from payment totals to connected accounts, compare how each model expresses the problem and how each database performs. Recommend which model suits each task and explain the evidence behind your investigation priorities.

In the supplied data, a Customer holds one or more Account wallets, whose roles are person, merchant or agent. A dated payment between accounts is a TRANSFER. Accounts also have observations linking them to a Device, such as a handset or public terminal. These observations do not identify which device initiated a payment.

This project requires up to 10 GB of available disk space. Use your installed software and keep the supplied mobile_money_generator.py unchanged. It yields records for your own program to capture. Complete the TODO gaps in the handout and write the remaining code. SQL placeholders %(name)s are Python psycopg parameters. Cypher uses $name.

Embed screenshots of each requested checkpoint in your report, with executed commands, results, taskbar and clock visible. Consolidate repeated cases into readable summary screenshots. Submit full outputs separately. Suggested pace is days 1 to 4 for loading, 5 to 9 for queries, and 10 to 14 for comparisons and reporting.

1 Capture the data

1.1 Prepare your workspace [2 marks]

1. Create a project PostgreSQL database and a dedicated Neo4j Community instance using its default neo4j database. Record software and driver versions, CPU, RAM, OS and database memory settings. Set PostgreSQL time zone to Africa/Kigali. Screenshot successful connections and settings.

1.2 Build capture.py [5 marks]

1. Import the generator as g. Inspect g.dataset_info() and g.FIELDS. Preview ten records from each iter_<stream>() using itertools.islice. Complete the starter below, then apply it to all five streams. Fresh iterator calls restart the stream.
2. Write CSV headers and rows progressively with csv.DictWriter. Count consumed and written rows, match the table counts below, log every million transfers, and record capture seconds and CSV sizes. Also accumulate transfer count, earliest/latest occurred_at and sum of amount_rwf. Screenshot the preview and completed five-stream totals.

Python capture starter

```
import csv
import mobile_money_generator as g
with open("transfers.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=g.FIELDS["transfers"])
    writer.writeheader()
    for row in g.iter_transfers():
        ...  # TODO write row, count it, update audit totals
```

Stream and table	Fields in CSV order	Rows
customers	customer_id, region	228,000
accounts	account_id, customer_id, role	248,000
devices	device_id, access_type	228,600
account_devices	account_id, device_id, first_day, last_day, observations	285,600
transfers	transfer_id, sender_id, receiver_id, occurred_at, amount_rwf	20,003,300

There are 20,993,500 source records for September 2026. occurred_at is Unix seconds and amount_rwf is integer RWF. Device intervals use inclusive day numbers 0 to 29, starting 1 September. observations is a count within that interval. Transfers are not globally time-sorted. Never collect the full stream in memory.

2 Build PostgreSQL

2.1 Define the five tables [4 marks]

1. First represent the captured records as the five PostgreSQL tables listed above. Use INTEGER for entity IDs, amounts and observations, BIGINT for transfer_id and occurred_at, SMALLINT for observation days, and TEXT for categories. Make every column NOT NULL. Set each entity/event ID as its primary key and (account_id, device_id) as the account_devices key.
2. Add foreign keys for ownership, device use and both transfer endpoints. Require positive amounts and observations, and 0 <= first_day <= last_day <= 29. Draw the schema and explain field meanings and units. Screenshot definitions and constraints.

2.2 Bulk-load and check [5 marks]

1. Load customers and devices, then accounts, account_devices and transfers. In psql, adapt the command below for each file. Record loading and index/constraint setup seconds separately.
2. Index accounts(customer_id, account_id) and account_devices(device_id, account_id). Index transfers(sender_id, occurred_at), INCLUDE receiver_id, amount_rwf, transfer_id. Also index (receiver_id, occurred_at), INCLUDE sender_id, amount_rwf, transfer_id. Add a BRIN index on occurred_at and run ANALYZE.
3. Match all row counts and transfer audit totals to capture.py. Confirm 240,000 person, 6,000 merchant and 2,000 agent accounts. Screenshot checks, validated constraints, indexes and database data/index sizes. Use 1 GB = 1,000,000,000 bytes throughout.

psql loading starter

```
\copy customers FROM '<TODO CSV path>' CSV HEADER
```

3 Build Neo4j

3.1 Map the same facts [4 marks]

1. Now represent the same records as a graph. Create Customer(customer_id, region), Account(account_id, role) and Device(device_id, access_type) node labels with integer IDs. Convert ownership into Customer-OWNS->Account and device observations into Account-USED_DEVICE->Device, retaining the three observation properties.
2. Make every transfer a separate Account-TRANSFER->Account relationship with transfer_id, occurred_at and amount_rwf. Draw the graph schema and map every source field to it. Preserve repeated payments between the same accounts.

3.2 Prepare and import CSVs [5 marks]

1. Create prepare_import.py. Make temporary import copies of the CSVs with Neo4j typed headers, retaining the source CSVs unchanged. Use :int or :long for numeric properties and separate Customer, Account and Device ID groups. Derive owns.csv in customer_id, account_id order.
2. Use the Account header example below. The ignored column remains in the CSV, while its ownership fact goes into owns.csv. Complete the import command with --id-type=integer and --nodes=Label=file.csv / --relationships=TYPE=file.csv options for all three node and three relationship files. Run full import with the empty project database stopped.
3. Start Neo4j. Create unique constraints on the three business IDs and a range index on TRANSFER.occurred_at. Wait for indexes to be online. Record import/setup seconds, store and transaction-log GB. Screenshot completion, constraints and indexes.
4. Verify node counts against the first three source streams, OWNS = 248,000, USED_DEVICE = 285,600 and TRANSFER = 20,003,300. Totals are 704,600 nodes and 20,536,900 relationships. Check account roles, one owner per account, unique transfer_id and the capture audit totals. Screenshot checks, then remove temporary import copies.

Account CSV header and import starter

```
account_id:ID(Account),:IGNORE,role:string
neo4j-admin database import full neo4j <TODO options>
```

3.3 Inspect one account in both models [3 marks]

1. Trace one loaded account to its owner, a device and an outgoing transfer using SQL JOINs and Cypher MATCH. Screenshot matching records. Explain how foreign keys become graph relationships and what event/time evidence would disappear if repeated payments were collapsed into one edge.

4 Answer routine business questions

Begin by measuring payment activity, then examine device sharing as a possible connection between accounts. Build one parameterized SQL/Cypher pair for each question Q1 to Q7 using stored data, not generator internals. For account-specific questions, use all six sets below. Each review account starts a search. Q7 alone uses the destination. Where an hour is specified, use 09:45:00 inclusive to 10:45:00 exclusive, CAT, on the listed date.

Set	Review account	Destination	September date
A	998	24755	6
B	147228	186823	13
C	206978	6573	8
D	237998	21755	6
E	144228	183823	13
F	203978	3573	8

4.1 Q1 Account activity [3 marks]

1. Complete and run the SQL and Cypher starters separately. Return outgoing count and total RWF in the specified hour, including all receiver roles, with zeros for no matches. Convert CAT boundaries to epoch seconds in Python. Screenshot all six paired results and explain why outgoing value is not a balance.

SQL and Cypher starters for the same question

```
SELECT COUNT(*), COALESCE(SUM(amount_rwf), 0)
FROM transfers WHERE sender_id = %(account)s
  AND occurred_at >= %(start)s AND ...; -- TODO end
MATCH (a:Account {account_id:$account})-[t:TRANSFER]->()
WHERE t.occurred_at >= $start AND ... // TODO end
RETURN count(t), ...; // TODO total with zero for no matches
```

4.2 Q2 Daily activity [3 marks]

1. Group the full month into CAT days using to_timestamp / AT TIME ZONE in SQL and datetime({epochSeconds: ..., timezone: "Africa/Kigali"}) in Cypher. Return date, count and total RWF, sorted by date. Match all 30 days and reconcile sums to capture totals.
2. Plot daily count and value with units. Screenshot results, checks and charts. Explain one pattern and why transfer value alone does not establish provider revenue.

4.3 Q3 Shared devices [4 marks]

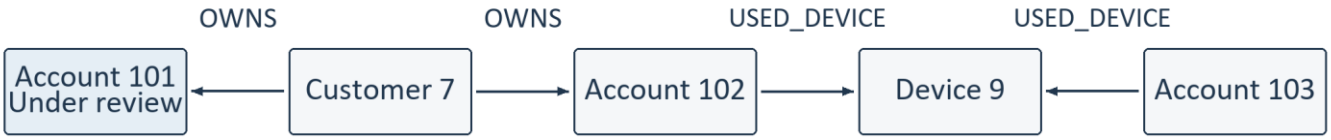
1. To find accounts observed using the same device during overlapping periods, self-join account_devices on device_id in SQL. In Cypher match Account-USED_DEVICE->Device<-USED_DEVICE-Account. Exclude the review account and include both device types. Keep overlaps where max(first_day values) <= min(last_day values).
2. Return other account ID, device ID, access_type, overlap start and end days, sorted by the first two fields. Screenshot matching results/counts for all sets. Explain a legitimate sharing scenario and why overlap does not prove simultaneous use. For example, intervals [2,7] and [5,11] overlap on days [5,7].

5 Investigate connected accounts

Device sharing gives a direct connection. Extend the investigation to indirect connections through ownership and devices, then follow transfer sequences. The diagrams illustrate the rules using teaching accounts. For each question, screenshot your query pair, completed case summary and the requested example from your loaded data.

5.1 Q4 Ownership and device network [6 marks]

1. Find distinct other accounts connected through OWNS or USED_DEVICE in either direction, within 4, 6, 8 and 10 links. Exclude transfers and apply no date/overlap filter. Count each reached account once and group by its owner region, sorted by region.
2. In SQL, build an edges CTE from ownership and device-use rows in both directions. Carry node kind and ID so Customer 7 differs from Account 7. Complete the recursive starter below. In Cypher use a bounded undirected path, then DISTINCT accounts before grouping.
3. Predict counts before testing depths 8 and 10. Screenshot all 24 comparisons, a count-versus-depth plot and one supporting path. Explain how deeper links can introduce weak associations.



Teaching example. Account 102 is two links from Account 101 through Customer 7. Account 103 is four links away through Device 9. Follow stored arrows in either direction for this association search.

Recursive SQL structure to complete

```
WITH RECURSIVE edges(sk, si, dk, di) AS (
  ... -- TODO ownership/device links in both directions
), walk(kind, id, depth) AS (
  SELECT 'Account', %(account)s::integer, 0
  UNION
  SELECT e.dk, e.di, w.depth + 1
  FROM walk w JOIN edges e ON ... -- TODO kind AND ID
  WHERE w.depth < %(depth)s
) SELECT ... FROM walk; -- TODO distinct accounts by region
```

Cypher starting pattern for Q4 at depth 4

```
MATCH (a:Account {account_id:$account})
MATCH (a)-[:OWNS|USED_DEVICE*1..4]-(b:Account)
WHERE ... // TODO exclude the starting account
RETURN ...; // TODO distinct accounts grouped by owner region
```

5.2 Q5 Rapid onward payments [5 marks]

1. Look for rapid onward payments by finding exactly three directed transfers through four distinct accounts. The first starts at the review account within its hour. Each later transfer occurs strictly after the preceding transfer and at most 600 seconds later. Later events may extend beyond the hour. Include all roles.
2. Each onward amount must be 90% to 105% of the preceding amount. Use 90*previous <= 100*next <= 105*previous, expressed as two comparisons with BIGINT arithmetic in SQL. Join three transfer aliases on receiver-to-sender, or match three directed Cypher relationships, then apply the rules.

3. Return ordered transfer IDs and final receiver, sorted by those fields. Screenshot all six matching counts and one chain with IDs, CAT times and amounts. Explain why this does not prove that the same funds moved throughout.



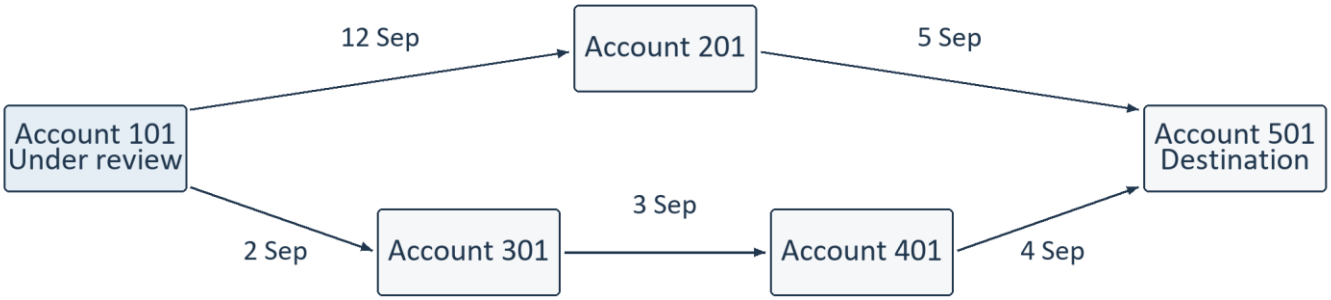
Teaching example. This chain qualifies. The onward amounts are 96% and 96.875% of the preceding amounts. Moving the final event to 10:15 CAT would violate the second ten-minute gap.

5.3 Q6 Circular payments [5 marks]

1. Now investigate whether a sequence of payments returns to the account where it began. Find 3 to 8 transfers starting and ending at the review account. All events must fall within its hour, in strictly increasing time order, spanning at most 1,800 seconds. Keep the Q5 amount rule and all roles. Stop at the first return. Other accounts may repeat. There is no 600-second gap rule.
2. Use recursive SQL carrying current account, ordered transfer IDs, first/last time and last amount. Extend only eligible transfers. Use a bounded Cypher path with equivalent checks. Return ordered transfer-ID sequences and lengths, sorted by sequence. Screenshot six matching counts and one cycle. State what further evidence you would request.

5.4 Q7 Shortest transfer connection [6 marks]

1. To assess how closely two accounts are connected in the transfer network, find the fewest directed transfers from each review account to its destination over the full month. Every account on the route must have role person. Ignore amounts and time order. Search with bounds 4 and 6, returning minimum length or no route within the bound.
2. Adapt the recursive approach to directed transfer neighbours in SQL. Use native shortest-path search in Neo4j. Apply the person restriction during search, not after choosing a shortest path. Separately retrieve one path for each successful case and verify IDs, direction, roles, event existence and length.
3. Screenshot all 12 length comparisons and path checks. Equally short paths may differ. Explain why no route within four transfers does not establish no connection, and why a structural route need not show onward movement of funds.



Teaching example. All shown accounts are personal. The upper route is shortest at two transfers even though its dates run backwards. The lower route uses three transfers.

6 Check correctness and performance

6.1 Build one comparison program [4 marks]

1. Before comparing speed, establish that both databases answer the same questions correctly. Write run_pg(query, params) and run_neo(query, params) functions returning every result row. Normalize integers and sort before exact comparison. Cover 61 cases across Q1 to Q7, respectively 6, 1, 6, 24, 6, 6 and 12. Compare Q7 lengths separately from valid supporting paths.
2. Resolve mismatches. Independently check one device overlap and one chain/cycle against captured CSV rows in Python. Screenshot the 61-case status table and these checks. Keep full outputs in the submission.

6.2 Time a fixed selection [5 marks]

1. Use the same machine, one database service at a time, persistent connections and fixed resource settings. Query base data without precomputed answers. Time 21 cases per engine. For Q1/Q3/Q5/Q6 use B and E, for Q2 use the full month, for Q4 use B/E at all four depths, and for Q7 use B/E at both bounds.
2. Run three untimed warm-ups then five measured runs per case. Time submission through final-row receipt using perf_counter. Allow 180 seconds per warm-up and 60 per measured query. Record errors/timeouts separately, never as completed timings.
3. Save case, parameters, engine, CAT start time, duration, row count and status. Report median/minimum/maximum milliseconds and PostgreSQL/Neo4j median ratio. Outside timing, collect EXPLAIN (ANALYZE, BUFFERS) and PROFILE for Q2, Q4 B depth 10 and Q7 B bound 6. Screenshot summaries/plans and explain one processing difference.

Timing loop after the three warm-ups

```
from time import perf_counter
durations = []
for _ in range(5):
    start = perf_counter()
    rows = ...  # TODO call one wrapper; consume all rows
    durations.append(perf_counter() - start)
# TODO save run details; compare answers outside the timer
```

6.3 Try one SQL improvement [5 marks]

1. A slow query may reflect its implementation as well as the database model. Use the Q7 plan to identify wasted search work, then predict and attempt one SQL improvement, such as deduplicating account-depth states, searching from both ends or stopping at a proven minimum. Keep both versions. Recheck all 12 cases and retime B/E at both bounds, giving four revised SQL cases.
2. Screenshot correctness, timings and before/after plans. Select a correct version for Section 7 and the final comparison using B/E bound-6 medians. Explain the outcome. A justified unsuccessful attempt can earn full marks. Never hard-code known answers.

7 Test concurrent requests

7.1 Build a small load test [5 marks]

1. The provider may have several analysts querying at once. Simulate this with Python multiprocessing, one persistent connection per process and a Barrier to synchronize starts. Test Q4 depth 10 and Q7 length-only bound 6 separately, cycling A-B-C with 1, 4, 8 and 16 clients in each engine. Use the SQL version selected in Section 6.

2. Each client warms up A-B-C once. At barrier release, set a common start time and a deadline 15 seconds later. Start cycles until that deadline, finish any started cycle, and submit each next request immediately after completion. Keep the 60-second query timeout. Repeat every engine/workload/client-count test three times.
3. Log each request and check correctness after timing. Throughput is correct completions divided by elapsed time from barrier release to the final cycle finishing. For each repetition report throughput, median latency, p95, errors and timeouts. p95 is sorted successful latency at position ceil(0.95*n), counting from 1. Summarize repetitions by their median. Screenshot active clients and all 16 combination summaries.

7.2 Explain scaling [4 marks]

1. Screenshot plots of throughput and p95 against client count for both workloads/engines. Compare 1 versus 16 clients and explain any saturation using resource and plan evidence. Distinguish throughput from individual response time. Assess whether a graph advantage survives SQL improvement, limiting claims to this experiment.

8 Make and defend decisions

8.1 Prioritize investigation [5 marks]

1. Use your findings to advise which accounts the provider should examine first. Compare A/D, B/E and C/F in three table rows using Q3 to Q7. For each pair, choose the account to review first or justify equal priority. Discuss one pair in detail using IDs, amounts, times and device evidence.
2. Show one transfer chain/cycle diagram and one ownership-device diagram from your results. Reuse Section 5 evidence and screenshots. Give an ordinary explanation for one suspicious association, missing evidence and how it could change your judgement. Do not declare fraud from these records.

8.2 Recommend a model [5 marks]

1. In at most 500 words, assign reporting and investigation work to PostgreSQL, Neo4j or both. Use three comparisons covering reporting, traversal and concurrency, plus load time/storage. Explain one SQL/Cypher and plan comparison. Include evidence against your preferred choice.
2. Distinguish graph modelling benefits from implementation/algorithm effects. If copying operational data into a graph, propose a freshness target and explain stale-data consequences. State two limits of your recommendation. No replication implementation is required.

8.3 Explain individual judgement [3 marks]

1. Each member writes 150 to 200 named words on a different measured finding, its explanation and a check that could overturn it. List contributions and any LLM/external assistance with how it was checked. Each member must be prepared to explain the work.

9 Submit reproducible work

9.1 Prepare the report [2 marks]

1. Submit one PDF in milestone order with names/IDs, models, summaries, charts, embedded evidence and individual notes. Keep explanations brief. Reference earlier screenshots for repeated evidence and keep complete outputs/raw repetitions in the code submission.

9.2 Provide code and verification [2 marks]

1. Submit a GitHub URL and matching ZIP with generator, capture/import programs, definitions, queries, checks, benchmarks, raw results and plans. Include README commands in execution order and placeholder secrets. Keep CSVs/database stores local. Verify the generator is unchanged.
2. Rerun the submitted 61-case comparison and screenshot completion. Stop the project database services and screenshot stopped status. Retain your stores. Sound findings of no graph advantage can earn full marks.

Feature guides

PostgreSQL recursive queries | Neo4j import headers | Neo4j full import | Cypher shortest paths